

Towards Automated Circuit Discovery for Mechanistic Interpretability

Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch,
Stefan Heimersheim, Adrià Garriga-Alonso

Presented by: **Anbuselvi S**

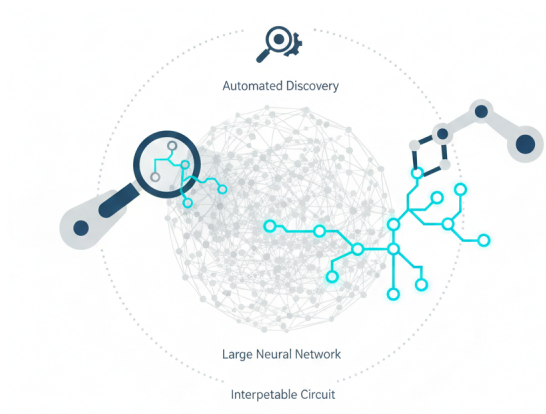
Presentation Outline

- Motivation and Background
- Circuits and Scalability Challenges
- Common Workflow for Circuit Discovery
- Automatic Circuit Discovery (ACDC)
 - behavior and dataset setup
 - computational graph representation
 - activation patching and recursive pruning
- Evaluation Methodology
- Related Work
- Limitations
- Conclusion

Motivation and Mechanistic Interpretability

- Rapid progress in **transformer language models** has led to new and complex emergent capabilities.
- Despite strong predictors of performance, transformers remain widely viewed as **black-box models**.
- Interpretability research seeks to:
 - demystify model behavior
 - explain outputs using domain-relevant concepts
- **Mechanistic interpretability** focuses on reverse-engineering model internals into **human-understandable algorithms**.

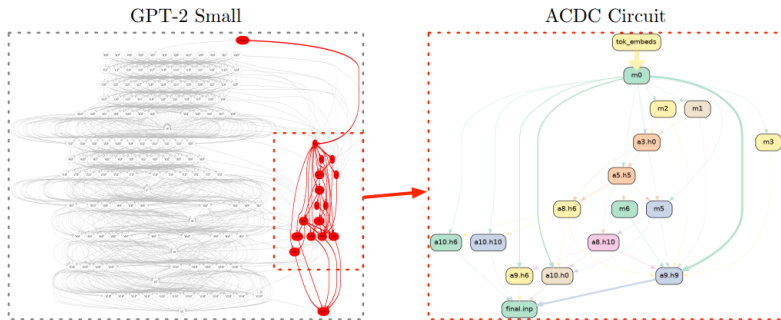
Can the discovery of behavior-specific circuits inside neural networks be automated rather than relying on manual mechanistic analysis?



Circuits and the Scalability Problem

- Model behaviors are implemented by algorithms within the model's **computational graph**, and **circuits** correspond to **subgraphs** that capture the majority of a specific behavior.
- Prior mechanistic interpretability work has successfully identified such circuits for several non-trivial behaviors.
- However, circuit discovery typically relies on **manual and iterative human inspection**.

Automatically Discovered Circuits with ACDC



Automatically discovering circuits with ACDC. Left: Computational graph for GPT-2 Small with the recovered IOI circuit highlighted in red (edges shown only between adjacent layers). Right: Recovered circuit with labeled nodes; all identified heads belong to the IOI circuit. Edge thickness denotes relative importance.

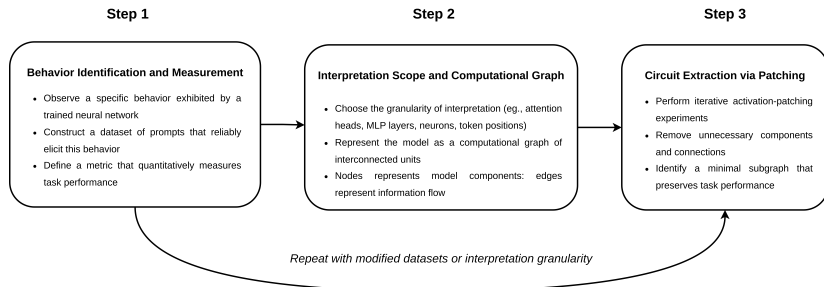
A Common Interpretability Workflow

- Prior mechanistic interpretability studies implicitly follow a similar procedure for circuit discovery.
- This work makes that procedure **explicit** by identifying a common workflow across studies.
- The workflow decomposes circuit discovery into **well-defined stages**, separating:
 - task and metric selection
 - model representation
 - circuit extraction
- Formalizing this workflow clarifies where interpretability effort is currently concentrated.

Mechanistic Interpretability Workflow

- Mechanistic interpretability studies across the literature follow a common **three-step workflow**.
- **Step 1:** Select a specific behavior and define an appropriate dataset and evaluation metric.
- **Step 2:** Represent the model as a **computational graph** at a chosen level of abstraction.
- **Step 3:** Identify a **circuit** by iteratively pruning components using activation patching.
- After circuit extraction, researchers interpret the functional roles of the remaining components.

Mechanistic Interpretability Workflow



A common three-step workflow for circuit discovery observed across mechanistic interpretability studies, highlighting the circuit extraction stage addressed in this work.

Step 1: Behavior, Dataset, and Metric

- Mechanistic interpretability begins by selecting a **specific model behavior** to analyze.
- A dataset is constructed to reliably **elicit this behavior** from the trained model.
- Alongside the main dataset, a **corrupted dataset** is defined to disrupt the behavior while preserving surface structure.
- A quantitative **metric** is used to measure behavioral strength and compare patched or pruned models.

Example: Greater-Than Task

- The **Greater-Than** task studies numerical comparison behavior in language models.
- Example prompt:
 - “The war lasted from 1517 to 15”
- The model assigns higher probability to completions greater than **17**.
- This task provides a clear, well-defined behavior for circuit discovery experiments.

Automatically Discovered Circuits with ACDC

Task	Example Prompt	Output	Metric
1: IOI (Appendix F.2)	“When John and Mary went to the store, Mary gave a bottle of milk to”	“_John”	Logit difference
2: Docstring (Appendix H.1)	<pre>def f(self, files, obj, state, size, shape, option): """document string example :param state: performance analysis :param size: pattern design :param</pre>	“_shape”	Logit difference
3: Greater-Than (Appendix G)	“The war lasted from 1517 to 15”	“18” or “19” or ... or “99”	Probability difference
4: tracr-xproportion (Appendix I.1)	[“a”, “x”, “b”, “x”]	[0, 0.5, 0.33, 0.5]	Mean Squared Error
5: tracr-reverse (Appendix I.2)	[0, 3, 2, 1]	[1, 2, 3, 0]	Mean Squared Error
6: Induction (Section 4.2)	“Vernon <i>Dursley</i> and Petunia <i>Durs</i> ”	“ley”	Negative log-probability

Five behaviors for which we have an end-to-end circuit from previous mechanistic interpretability work, plus Induction. We automatically rediscover the circuits for behaviors 1-5 in Section 4.

Tokens beginning with space have a “ ” prepended for clarity.

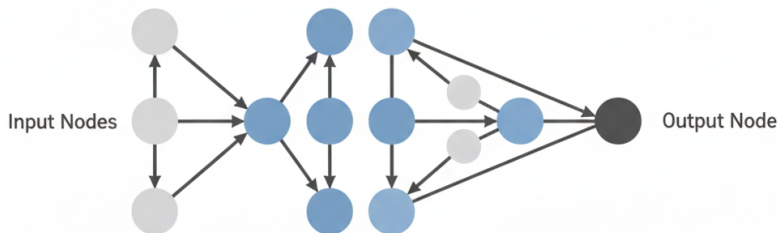
Step 2: Computational Graph Representation

- To identify circuits, the model is represented as a **directed acyclic computational graph**.
- Nodes correspond to model components, and edges represent the **flow of information**.
- The choice of graph abstraction determines the **granularity** of the resulting circuit.
- This representation defines the search space for circuit extraction.

Step 2: Levels of Abstraction

- Computational graphs can be defined at different levels of abstraction.
- Examples include:
 - attention heads and MLP layers
 - query, key, and value paths
 - individual neurons
 - token-position-specific nodes
- Finer abstractions yield more detailed circuits but increase computational cost.

Computational Graph Illustration



Schematic representation of a directed acyclic computational graph, where nodes denote model components and edges represent information flow toward the output.

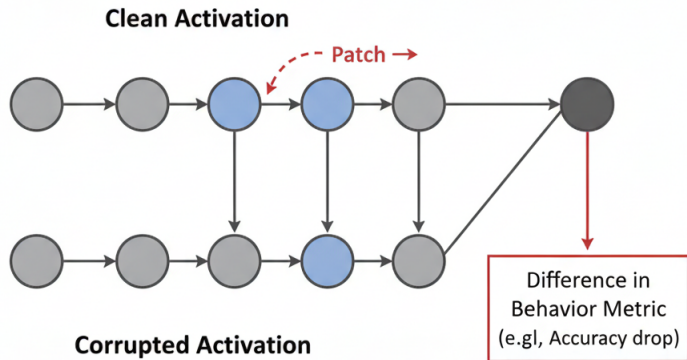
Step 3: Activation Patching

- Circuit extraction proceeds by **activation patching** on the computational graph.
- Activations of nodes or edges are replaced with values from a **corrupted dataset**.
- The model is evaluated using the predefined metric to measure behavioral change.
- Components whose removal significantly degrades performance are considered important for the behavior.

Step 3: Recursive Circuit Extraction

- Circuit discovery begins at the **output node** of the computational graph.
- Incoming edges are tested individually using activation patching.
- Edges with negligible impact on the metric are pruned from the graph.
- This process is applied **recursively**, traversing backward through the graph.

Activation Patching Mechanism

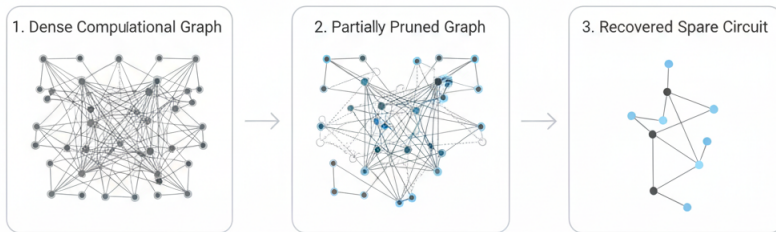


Activation patching compares model behavior under clean and corrupted activations by replacing internal values and measuring changes in the task-specific behavior metric.

Automatic Circuit Discovery (ACDC)

- **Automatic Circuit Discovery (ACDC)** is proposed to automate circuit extraction in mechanistic interpretability.
- Prior work relied on **manual, iterative** activation patching guided by human intuition.
- ACDC systematizes this process by automatically testing and pruning connections in the computational graph.
- The method outputs a **sparse circuit** that preserves performance on a chosen behavior.

Circuit Discovery through Pruning



Progressive pruning of a dense computational graph into a sparse circuit, illustrating the objective of circuit discovery methods such as ACDC.

ACDC: Algorithmic Overview

- ACDC operates on a fixed computational graph and a predefined behavior metric.
- Starting from the **output node**, incoming edges are evaluated using activation patching.
- An edge is removed if replacing its activation with corrupted values does not significantly change the metric.
- The procedure proceeds **recursively in reverse topological order** until a minimal circuit is recovered.

ACDC Algorithm (Formal Reference)

Algorithm 1: The ACDC algorithm.

Data: Computational graph G , dataset $(x_i)_{i=1}^n$, corrupted datapoints $(x'_i)_{i=1}^n$ and threshold $\tau > 0$.

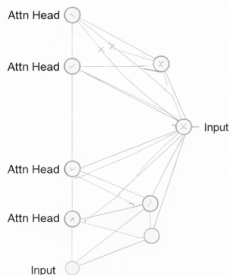
Result: Subgraph $H \subseteq G$.

```
1  $H \leftarrow G$  // Initialize H to the full computational graph
2  $H \leftarrow H.reverse\_topological\_sort()$  // Sort H so output first
3 for  $v \in H$  do
4   for  $w$  parent of  $v$  do
5      $H_{new} \leftarrow H \setminus \{w \rightarrow v\}$  // Temporarily remove candidate edge
6     if  $D_{KL}(G||H_{new}) - D_{KL}(G||H) < \tau$  then
7        $H \leftarrow H_{new}$  // Edge is unimportant, remove permanently
8     end
9   end
10 end
11 return  $H$ 
```

Formal description of the Automatic Circuit Discovery (ACDC) algorithm.

Automatic Circuit Discovery: Greater-Than Task

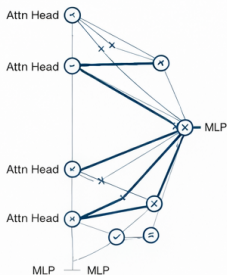
Full Computational Graph



Edges removed based on metric preservation

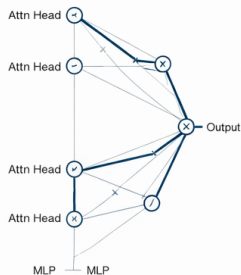
All 5 component types rediscovered

ACDC Edge Evaluation and Pruning



Edges removed based on metric preservation

Final Discovered Circuit



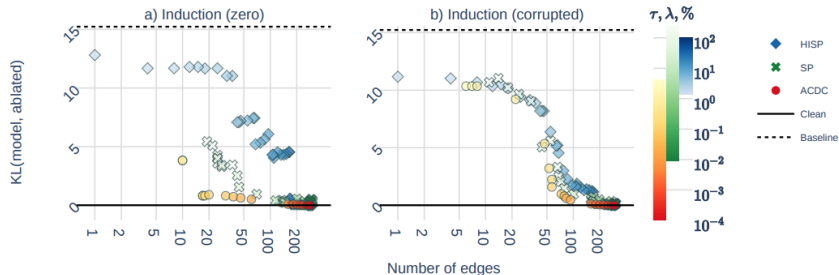
Minimal edges retained

Evaluation Objectives

- The evaluation focuses on assessing the **quality of circuits** recovered by ACDC.
- Two primary questions are investigated:
 - **Q1:** Does ACDC recover circuits that preserve task performance?
 - **Q2:** How do recovered circuits compare to those obtained by prior methods?
- Evaluation is performed across multiple tasks and levels of abstraction.

- ACDC is compared against prior circuit discovery and pruning approaches.
- **Subnetwork Probing (SP):**
 - Identifies subnetworks by optimizing masks over model components.
- **Head Importance Score for Pruning (HISP):**
 - Prunes attention heads based on importance scores.
- These methods serve as reference points for circuit recovery quality.

Evaluation: Performance–Sparsity Trade-off



KL divergence versus number of edges for circuits recovered on the Induction task. ACDC (red) achieves lower divergence with fewer edges compared to SP and HISP, under both zero and corrupted activation patching.

Evaluation: Recovery vs. Prior Circuits

- Circuit discovery is evaluated by comparison with canonical circuits reported in prior mechanistic interpretability studies.
- Circuit recovery is formulated as a **binary classification problem** over edges:
 - edge belongs to the circuit
 - edge does not belong to the circuit
- Performance is summarized using **ROC curves** and the **Area Under the Curve (AUC)**.
- High true-positive rates indicate recovery of known circuit components (Q1), while low false-positive rates reflect exclusion of irrelevant components (Q2).

Evaluation: Stand-alone Circuit Properties

- To avoid reliance on imperfect human-annotated circuits, the authors also evaluate intrinsic properties of the recovered subgraphs.
- Circuit quality is assessed using:
 - similarity of circuit behavior to the original model
 - number of edges retained in the circuit
- A high-quality circuit preserves the target behavior while remaining sparse.
- This evaluation provides indirect evidence for both recovery quality (Q1) and exclusion of irrelevant components (Q2).

High-Level Evaluation Findings

- ACDC successfully recovers sparse circuits that preserve task performance across evaluated tasks.
- In several settings, ACDC identifies **smaller or comparable circuits** relative to baseline methods.
- Circuit quality is sensitive to:
 - choice of behavior metric
 - corrupted dataset construction
 - graph abstraction level

Limitations and Scope

- ACDC does not automate the **interpretation** of recovered circuits.
- Results depend on:
 - the selected behavior and dataset
 - the abstraction of the computational graph
- Circuit discovery is performed **post-hoc** on trained models.
- The method does not provide guarantees of uniqueness or causal correctness.

Conclusion

- This work systematizes a common workflow for mechanistic interpretability.
- ACDC automates circuit extraction using activation patching and recursive pruning.
- The approach scales circuit discovery beyond manual analysis.
- Automating circuit interpretation remains an open research direction.

Mathematical Foundations (Appendix)

- The appendix formalizes the mathematical foundations that underpin the Automatic Circuit Discovery (ACDC) framework.
- Circuit quality is quantified using the **Kullback–Leibler (KL) divergence** between the output distribution of the original model and that of the pruned subgraph.
- Edge pruning is governed by a **threshold-based criterion**, where edges are removed if their contribution induces only a negligible change in the evaluation metric.
- Additional analyses include:
 - alternative evaluation metrics (e.g., logit difference)
 - comparison between zero and corrupted activation patching
 - robustness checks and identified failure cases
- These results collectively support both the correctness and the documented limitations of the proposed method.

Backup: ACDC Pruning Criterion

An edge $(w \rightarrow v)$ is removed if:

$$\text{DKL}(G \parallel H_{\text{new}}) - \text{DKL}(G \parallel H) < \tau$$

where:

- G is the full model
- H is the current subgraph
- τ is a sparsity threshold

How ACDC Works (Summary)

- Step a: specify task, computational graph, and threshold τ .
- Step b: evaluate each connection by activation patching.
- Step c: prune connections with metric impact below τ .
- Recursively apply until the full circuit is recovered.

ACDC returns a minimal subgraph that preserves task performance.

Thank You